SMIR: Spectro-Magnetic Intermediate Representation (SMIR)

Technical Spec v0.1 (with external references)

0. Purpose

SMIR is a typed intermediate representation and compilation stack for **hybrid programs** that combine:

1. classical digital computation,
2. open-system quantum/control dynamics (NMR-realistic), and
3. instrument-bound spectral/analog kernels.

The central design goal is **predictive compilation**: mapping programs onto heterogeneous backends while tracking **precision, latency, and energy** budgets, and producing implementable control waveforms (e.g., RF pulses) and spectral acquisition plans.

Non-goals

- Not a proposal for scalable fault-tolerant quantum computing (NMR/QIP limitations are well known) [R13–R14].
- Not a replacement for digital computation; SMIR targets **narrow primitives** where physics provides a throughput/energy edge.

---

1. Core abstraction

SMIR treats three state families as **first-class** and composable:

- **Classical**: discrete arrays/scalars used for control flow, orchestration, and digital post-processing (DSP foundations) [R19–R20].
- **Quantum/Open**: density operators and quantum channels reflecting realistic device physics (open quantum systems) [R1–R4].
- **Spectrum/Instrumented**: frequency-domain objects tied to a measurement chain (bandwidth, resolution, noise, calibration) [R6, R19].

A SMIR program is a directed acyclic graph (DAG) of typed operators with explicit **resource annotations** and optional **calibration contracts**.

---

2. Type system

2.1 Primitive types

- `Classical[T, N]`
- `T ∈ {Real, Complex, Int, Bool}`
- `N`: shape (scalar if omitted).
- Examples: `Classical[Real, N]`, `Classical[Complex, N]`, `Classical[Real]`.
- `QState[H]`
- Density operator over Hilbert space descriptor `H` (e.g., `H = (qubits=k, topology=..., spins=...)`).
- `Channel[H_in → H_out]`
- CPTP map; may carry a parameter struct and a physical realization tag.
- `Spectrum[BW, Res, Inst]`
- Frequency-domain object with bandwidth `BW`, resolution `Res`, tied to an instrument configuration `Inst`.
- `Inst` is a nominal identifier: e.g., `Inst = NMR_Probe_1` or `RF_CMAC_v0`.
- `Waveform[Fs, T, Domain]`
- Time-domain control waveform (digital baseband or RF). `Fs` sample rate, `T` duration.

2.2 Effect and resource annotations

Every operator may carry a `ResourceTag`:

- `duration` (seconds)
- `energy` (joules, or proxy units)
- `precision_enob` (effective number of bits)
- `noise_model` (symbolic ID + parameters)
- `calibration_state` (required calibration version)

SMIR supports optional **uncertainty types**:

- `Classical[T] ± ε` meaning a value with an error bound or confidence interval.

---

3. Operator set (minimal v0)

3.1 Classical operators

- `add`, `mul`, `matmul`, `fft`, `ifft`, `argmax`, `normalize`, `window`, `threshold` [R19].

3.2 Quantum/Open operators

- `lift_diag(x: Classical[Real, N]) -> QState[H]`
  Creates a diagonal density operator from classical probabilities or populations.

- `apply(E: Channel[H→H], ρ: QState[H]) -> QState[H]`
  Applies a CPTP channel [R4].

- `measure(M: POVM[H], ρ: QState[H]) -> (Classical[Real, K] ± ε, QState[H])`
  Returns outcome statistics (or samples) and post-measurement state [R4].

- `compile_control(plan: ControlPlan) -> Waveform[...]`
  Lowers an abstract control plan into an implementable waveform; pulse-level control is a first-class compilation target in practice [R7–R8].

3.3 Spectral/Instrumented operators

- `acquire(ρ: QState[H], setup: AcquisitionSetup[Inst]) -> Spectrum[BW, Res, Inst]` (NMR acquisition concepts) [R6].

- `filter(S: Spectrum[...], K: KernelSpec) -> Spectrum[... ]` [R19].

- `downconvert(S: Spectrum[...], lo: Waveform[...]) -> Spectrum[...]` (heterodyne/mixing as a computational primitive) [R15–R16].

- `correlate(S1: Spectrum[...], S2: Spectrum[...]) -> Classical[Real] ± ε`
  Instrumented correlation/matched filter (radar/DSP context) [R18–R20].

3.4 Spectral CMAC primitive

- `spectral_dot(a: Classical[Complex, N], b: Classical[Complex, N], budget: NoiseBudget, inst: Inst) -> Classical[Complex] ± ε`

Semantics: returns an approximation to $\langle a, b \rangle$ subject to instrument constraints and a declared noise/precision budget. Correlator-style devices exist in RF/photonic signal processing as real-time correlators [R16], and analog signal processing frameworks explicitly target time-frequency primitives such as compression/correlation/discrimination [R15].

---

4. Formal semantics (predictive model)

4.1 Quantum/Open dynamics

SMIR backends interpret `Channel` either:

- by explicit Kraus operators $\{K_i\}$: $\mathcal{E}(\rho) = \sum_i K_i \rho K_i^\dagger$ [R4], or
- by Lindblad evolution for time-parameterized channels (GKS–Lindblad generators) [R1–R3]:

$$ \frac{d\rho}{dt} = -\frac{i}{\hbar}[H(t),\rho] + \sum_j \gamma_j\left(L_j\rho L_j^\dagger - \tfrac12{L_j^\dagger L_j,\rho}\right). $$

4.2 Spectral objects

A `Spectrum` denotes a random variable generated by an instrument chain:

$$ S(\omega) = \mathcal{M}_{Inst}(\rho;\text{setup}) + \eta(\omega) $$

where $\mathcal{M}_{Inst}$ encodes response functions, transfer functions, quantization, and calibration parameters, and η is noise (thermal + electronics + quantization). This is consistent with practical spectroscopy/NMR modeling where acquisition and instrument response are inseparable from the observable [R6].

4.3 Resource semantics

Every physical operator induces constraints:

- **Time**: must fit within coherence/relaxation windows (e.g., NMR T_2) [R5–R6].
- **Precision**: determined by ENOB, averaging, bandwidth, and calibration error.
- **Energy**: estimated from waveform power + acquisition chain.

The compiler is permitted to insert:

- calibration cycles,
- averaging loops,
- digital error mitigation,
- rescaling and windowing, provided overall constraints remain satisfied.

---

5. Backends

5.1 Digital backend

- Executes classical ops.
- Simulates `Channel` and `Spectrum` for verification and compiler cost modeling.

5.2 Spectral CMAC backend (analog/spectral accelerator)

Target operation: high-throughput complex dot products / correlations.

Microarchitecture sketch (v0)

1. **Encoding**: DAC maps vectors to multi-tone combs or spread codes:
2. $a \mapsto s_a(t), b \mapsto s_b(t)$
3. **Kernel**: analog mixing computes $s_a(t)\,\overline{s_b(t)}$ (multiplication → correlation primitives) [R15–R16]
4. **Accumulate**: integrate-and-dump or filter bank accumulates coefficients
5. **Digitize**: ADC produces output(s); digital post-processing debiases via calibration model.

Compiler contract

- Must respect bandwidth, PAPR constraints, and tone spacing.
- Returns $\hat{y} = \langle a, b \rangle + \epsilon$ with ϵ bounded/characterized by `NoiseBudget`.

5.3 NMR backend (open-system control + sensing)

Compiles `ControlPlan` to pulse sequences and acquisition setups.

Channel library (minimal)

- `pulse(axis, angle, width, amp, phase, σ_cal) : Channel[H→H]` (pulse imperfections and calibration are central in practice) [R5, R7–R8]
- `evolve(H_drift, t) : Channel[H→H]` [R5–R6]
- `relax(T1, T2, t) : Channel[H→H]` (open system) [R3]
- `gradient(G, t) : Channel[H→H]` (optional imaging/suppression) [R6]

Acquisition

- `acquire(ρ, setup)` produces `Spectrum[...]` representing FID/FFT with instrument response [R6].

---

6. Compilation pipeline

SMIR is designed to interoperate with modern heterogeneous compiler infrastructure (LLVM/MLIR) and quantum IR ecosystems (OpenQASM/QIR) [R9–R12].

1. **Front-end lowering**: user DSL / Python bindings → SMIR graph.
2. **Type + resource inference**: propagate bandwidth/resolution, timing windows, and target ENOB.
3. **Partitioning**: assign ops to Digital / Spectral / NMR backends via cost model.
4. **Schedule**: produce an execution plan with explicit calibration checkpoints.
5. **Codegen**:
6. Digital kernels (CPU/GPU),
7. Spectral CMAC control/config (DAC waveforms, LO settings, filter bank config),

8. NMR pulse sequences + acquisition parameters.
9. **Verification** (optional): simulate with noise models; compare to resource constraints.

---

7. Calibration & correctness

SMIR correctness is defined **relative to a calibration state**.

- Each physical backend exposes `CalState` with versioned parameters (gain/phase, transfer functions, drift models).
- Operators declare `requires CalState ≥ v`.
- Compiler may insert `calibrate(inst) -> CalState'` steps.

Outputs from physical operators are treated as **stochastic**; programs can request:

- bounds: $\|\epsilon\| \leq \epsilon_0$, or
- confidence: $P(|\epsilon| \leq \epsilon_0) \geq 1 - \delta$.

---

8. Reference benchmark: Batched pulse compression (matched filtering)

Problem: Given received signal `r` (length N) and M template pulses t_i, compute correlation peaks: $\text{score}(i) = \max_k |(r * \overline{t_i})(k)|$.

This benchmark is canonical in radar/DSP texts and matched filtering literature [R18–R20].

Digital baseline

- FFT convolution per template: $O(MN \log N)$ [R19].

SMIR hybrid program (v0)

1. Digital: preprocess `r` (window/normalize).
2. Spectral: encode `r` once; for each `t_i` stream:
3. `spectral_dot(r, t_i)` (or blockwise spectral correlation),
4. digital peak selection.
5. Output: `argmax` over templates.

Success metric

- Better **throughput-per-Watt** (or latency) than digital baseline at fixed detection reliability.
- SMIR-predicted error bounds must preserve correct template ranking with high probability.

Related implementation contrasts (digital vs. analog/SAW matched filtering) are documented in pulse compression practice [R17].

9. Deliverables (v0 → v1)

v0 (fast validation)

- SMIR graph IR + interpreter with noise/resource propagation.
- Digital simulator for `spectral_dot` with configurable ENOB, bandwidth, drift.
- Benchmark harness for batched pulse compression.

v1 (hardware-in-the-loop)

- SDR-based prototype of `spectral_dot` (signal generator/mixer/filter/ADC).
- Calibration routine + measured ENOB/latency/energy.
- Optional: benchtop NMR pulse/acquire demo showing channel compilation and spectrum prediction.

---

10. Expected outcomes

- Demonstrable advantage on a narrow class of bilinear signal kernels.
- A reproducible methodology for **predictive compilation under physical noise**.
- Clear boundaries on when analog/spectral acceleration helps (and when it does not).

---

11. External references

Open quantum systems / channels

- **[R1]** G. Lindblad, “On the generators of quantum dynamical semigroups,” *Communications in Mathematical Physics* **48**, 119–130 (1976). DOI: 10.1007/BF01608499.
- **[R2]** V. Gorini, A. Kossakowski, E. C. G. Sudarshan, “Completely positive dynamical semigroups of N-level systems,” *Journal of Mathematical Physics* **17**, 821–825 (1976). DOI: 10.1063/1.522979.
- **[R3]** H.-P. Breuer and F. Petruccione, *The Theory of Open Quantum Systems*. Oxford University Press (2002). ISBN: 9780198520634.
- **[R4]** M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press (2000). ISBN: 0521635039.

NMR / control

- **[R5]** M. H. Levitt, *Spin Dynamics: Basics of Nuclear Magnetic Resonance* (2nd ed.). Wiley (2008). ISBN: 0470511176.
- **[R6]** R. R. Ernst, G. Bodenhausen, A. Wokaun, *Principles of Nuclear Magnetic Resonance in One and Two Dimensions*. Oxford University Press (1987). ISBN: 9780198556473.
- **[R7]** N. Khaneja et al., “Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms,” *Journal of Magnetic Resonance* **172**(2), 296–305 (2005). DOI: 10.1016/j.jmr.2004.11.004.

- **[R8]** D. C. McKay et al., “Qiskit Backend Specifications for OpenQASM and OpenPulse Experiments,” arXiv:1809.03452 (2018).

Quantum IRs / compiler infrastructure

- **[R9]** A. W. Cross et al., “OpenQASM 3: A Broader and Deeper Quantum Assembly Language,” *ACM Transactions on Quantum Computing* **3**(3) (2022). DOI: 10.1145/3505636.
- **[R10]** Microsoft Azure Quantum, “Quantum Intermediate Representation (QIR)” (documentation; updated 2025).
- **[R11]** C. Lattner and V. Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation,” *CGO 2004* (IEEE) (2004). DOI: 10.1109/CGO.2004.1281665.
- **[R12]** C. Lattner et al., “MLIR: A Compiler Infrastructure for the End of Moore’s Law,” arXiv:2002.11054 (2020).

NMR quantum information (context/limitations)

- **[R13]** D. G. Cory, A. F. Fahmy, T. F. Havel, “Ensemble quantum computing by NMR spectroscopy,” *PNAS* **94**(5), 1634–1639 (1997). DOI: 10.1073/pnas.94.5.1634.
- **[R14]** N. A. Gershenfeld and I. L. Chuang, “Bulk spin-resonance quantum computation,” *Science* **275**, 350–356 (1997).

Analog/spectral processing and correlators

- **[R15]** C. Caloz et al., “Analog Signal Processing,” arXiv:1307.2618 (2013).
- **[R16]** G. Bourdarot, J.-P. Berger, H. Guillet de Chatellus, “Multi-delay photonic correlator for wideband RF signal processing,” *Optica* **9**(4), 325–334 (2022). DOI: 10.1364/OPTICA.442906.

Matched filtering / pulse compression

- **[R17]** P. Tortoli, F. Guidi, C. Atzeni, “Digital vs. SAW matched filter implementation for radar pulse compression,” *Proc. IEEE Ultrasonics Symposium* (1994), pp. 199–202.
- **[R18]** M. A. Richards, *Fundamentals of Radar Signal Processing* (3rd ed.). McGraw-Hill (2022). ISBN: 9781260468717.
- **[R19]** A. V. Oppenheim, R. W. Schafer, J. R. Buck, *Discrete-Time Signal Processing* (2nd ed.). Prentice-Hall (1999). ISBN: 978-0-13-754920-7.
- **[R20]** J. G. Proakis and M. Salehi, *Digital Communications* (5th ed.). McGraw-Hill (2007).